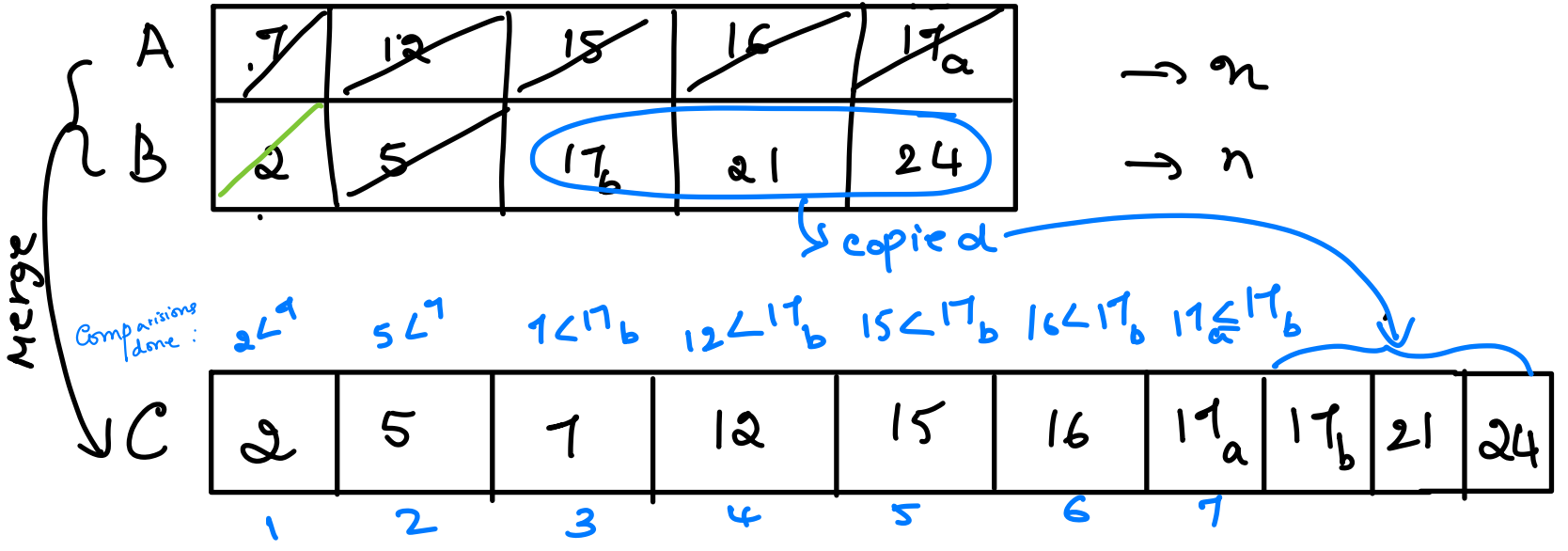
Result:

Merging two (already) sorted arrays to a single sorted array takes linear time.

Merging two Sorted Arrays:



n times compared.

Q : T(n) = ?

Prof. Kedukodi Babushri Srinivas, Mathematics, SOMAC, MIT Manipal

Merging two sorted arrays:

Merge $(A[1, \overset{i\leq}{\ldots}, p], B[1, \overset{j\leq}{\ldots}, q], C[1, 2, \ldots, p+q])$

$i \leftarrow 1, \quad j \leftarrow 1, \quad k \leftarrow 1;$

while $i \leq p$; $j \leq q$

If $A[i] \leq B[j]$ ← basic operation

$C[k] \leftarrow A[i]$; $i \leftarrow i+1$ ⇝ does the exit of while loop when endpn q is reached

else $C[k] \leftarrow B[j]$; $j \leftarrow j+1$

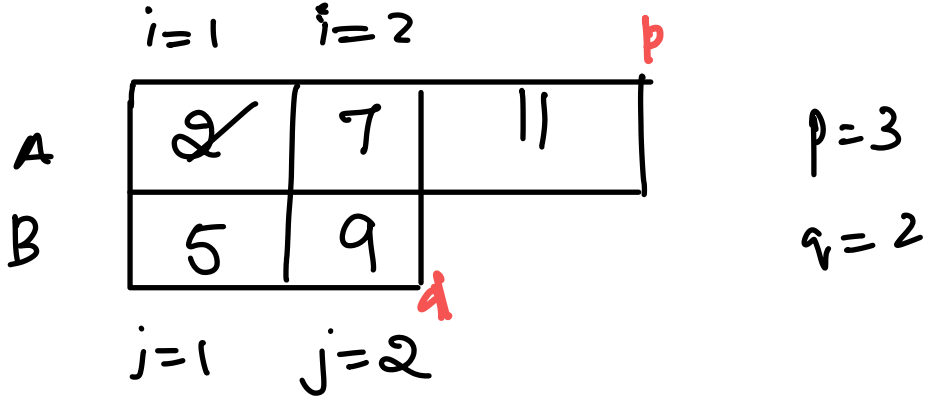
$k \leftarrow k+1.$

If $i = p+1$ then copy $B[j \cdots q]$ to $C[k \cdots p+q]$

If $j = q+1$ then copy $A[i \cdots p]$ to $C[k \cdots p+q]$

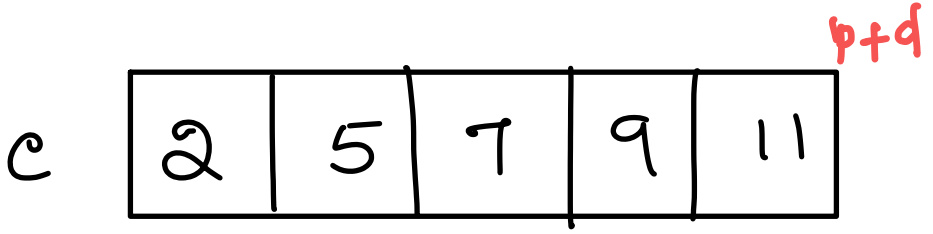
Analysis:

eg:



$p=3$

$q=2$



$k=1,2,3,4,5$

4 comparisions

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

* Each value of k involves one comparision except the last $(p+q)^{th}$ value

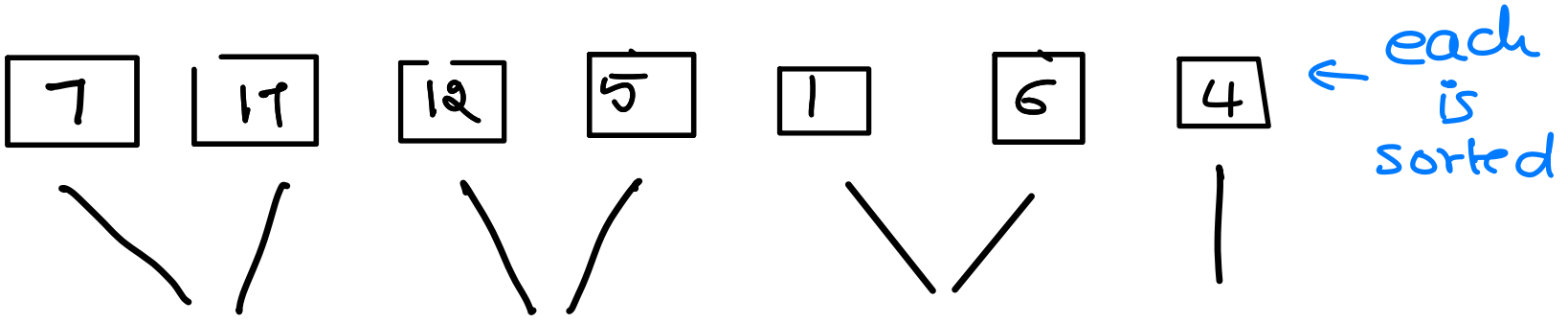
$\therefore$ Total comparisions $= p+q-1$.

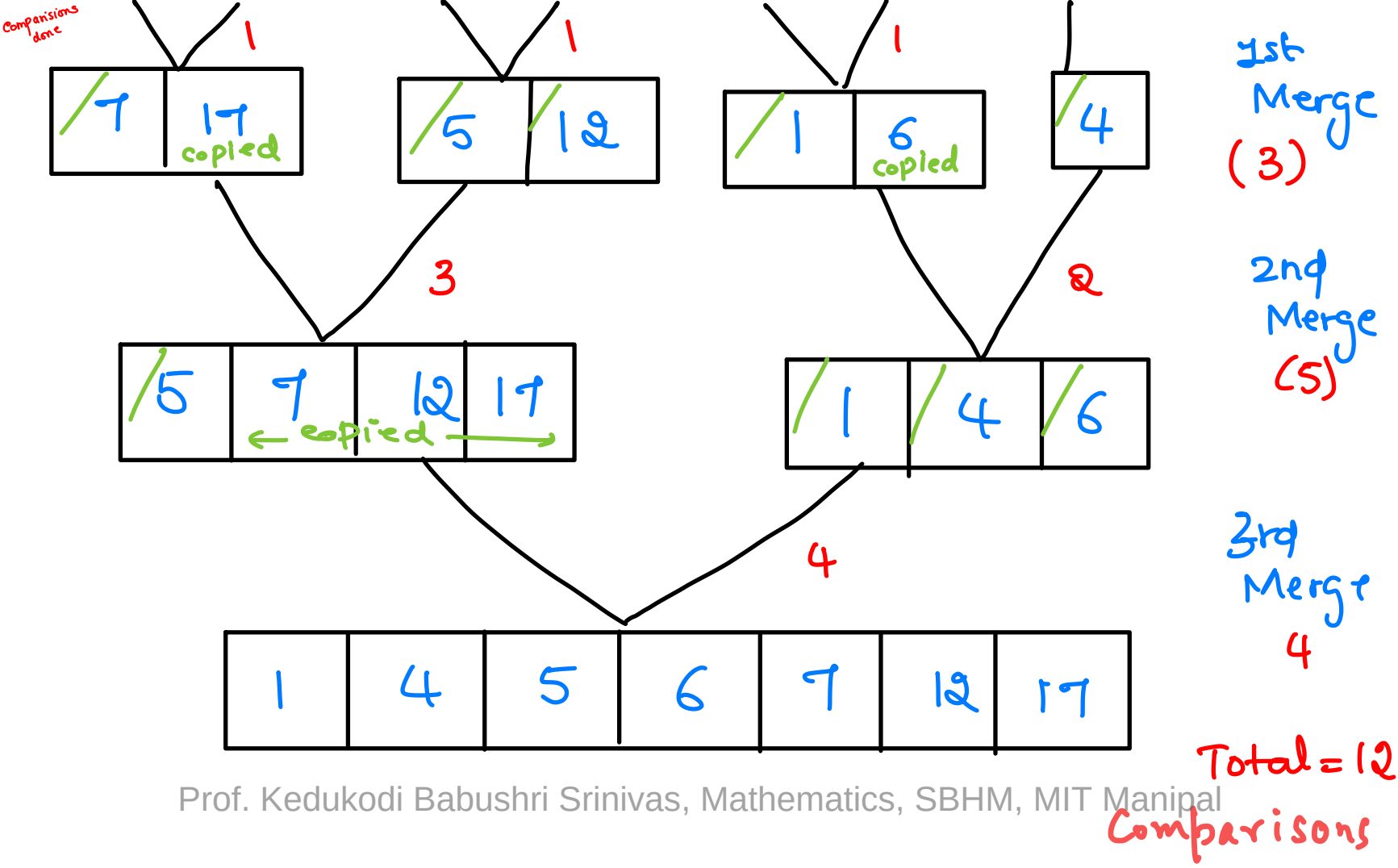
Let $p=q=n$

$T(n) = 2n-1 \in \Theta(n)$.

Merge Sort:

7	17	12	5	1	6	4




Comparisons done
7
17
copied
1
5
12
1
1
6
copied
1
4
1st Merge (3)
3
2
2nd Merge (5)
5
7
12
17
copied
1
4
6
4
3rd Merge 4
1
4
5
6
7
12
17
Total = 12 Comparisons

Q: How many comparisons are done in the worst case?

Merge Sort Algorithm:

Merge Sort $(A[1 \cdots n])$

If $n > 1$

copy $A[1 \cdots \lfloor \frac{n}{2} \rfloor - 1]$ to $B[1 \cdots \lfloor n/2 \rfloor - 1]$

copy $A[\lfloor \frac{n}{2} \rfloor \cdots n]$ to $C[1 \cdots \lfloor \frac{n}{2} \rfloor - 1]$

Merge Sort $(B[1 \cdots \lfloor \frac{n}{2} \rfloor - 1])$

Mutharasu Babushri Srinivas; Mathematics, SBHM, MIT Manipal

Merge Sort $(C[1 .. \lfloor \frac{n}{2} \rfloor - 1])$

Merge (B, C, A)

* These type of divide-and-Conquer algorithms are often analysed using recurrence relations (or equations).

Recurrence Equation:

$$T(n) = \begin{cases} \Theta(1) & , n = 1 \\ 2T(n/2) + \Theta(n) & , n > 1 \end{cases}$$

$\Theta(n)$: merging time (linear) cn

$2T(n/2)$: each time we get 2 problems of size half the initial one

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

$$T(n) = 2T(n/2) + cn \quad \leftarrow \theta(n) = cn$$

$$= 2\left(2T(n/4) + cn/2\right) + cn$$

$$= 2^2 T(n/4) + cn + cn$$

$$= 2^2\left(2T(n/8) + cn/4\right) + cn + cn$$

$$= 2^3 T(n/8) + cn + cn + cn$$

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

$$= 2^3 T(n/8) + c(3n)$$

$$\vdots$$

$$= 2^x T(1) + c(xn)$$

$$= 2^{\log_2 n} + c(\log n)n$$

$$= n + c(n \log n)$$

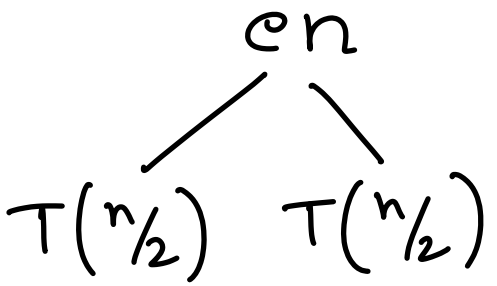
Prof. Reddukodi Babusri Srinivas, Mathematics, SBHM, MIT Manipal

$\in O(n \log n)$

($\because$ $n \log n$ is greater than n)

OR

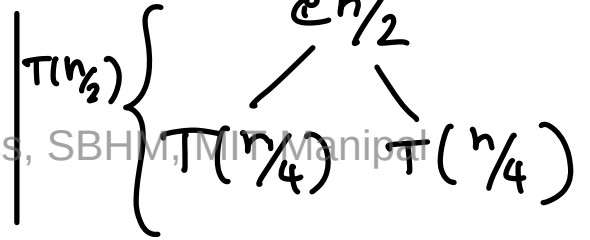
$$T(n) = 2T(n/2) + cn. \quad - (*)$$



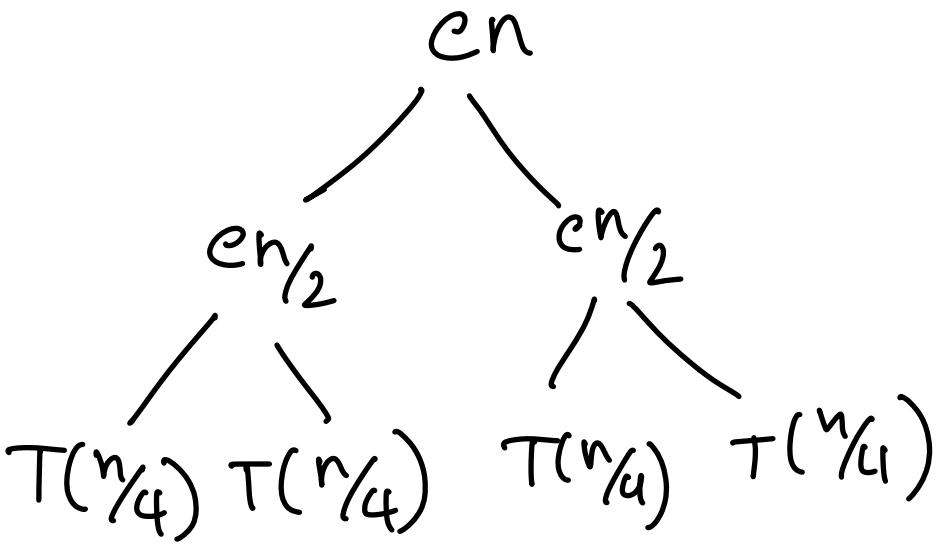
$\} T(n)$

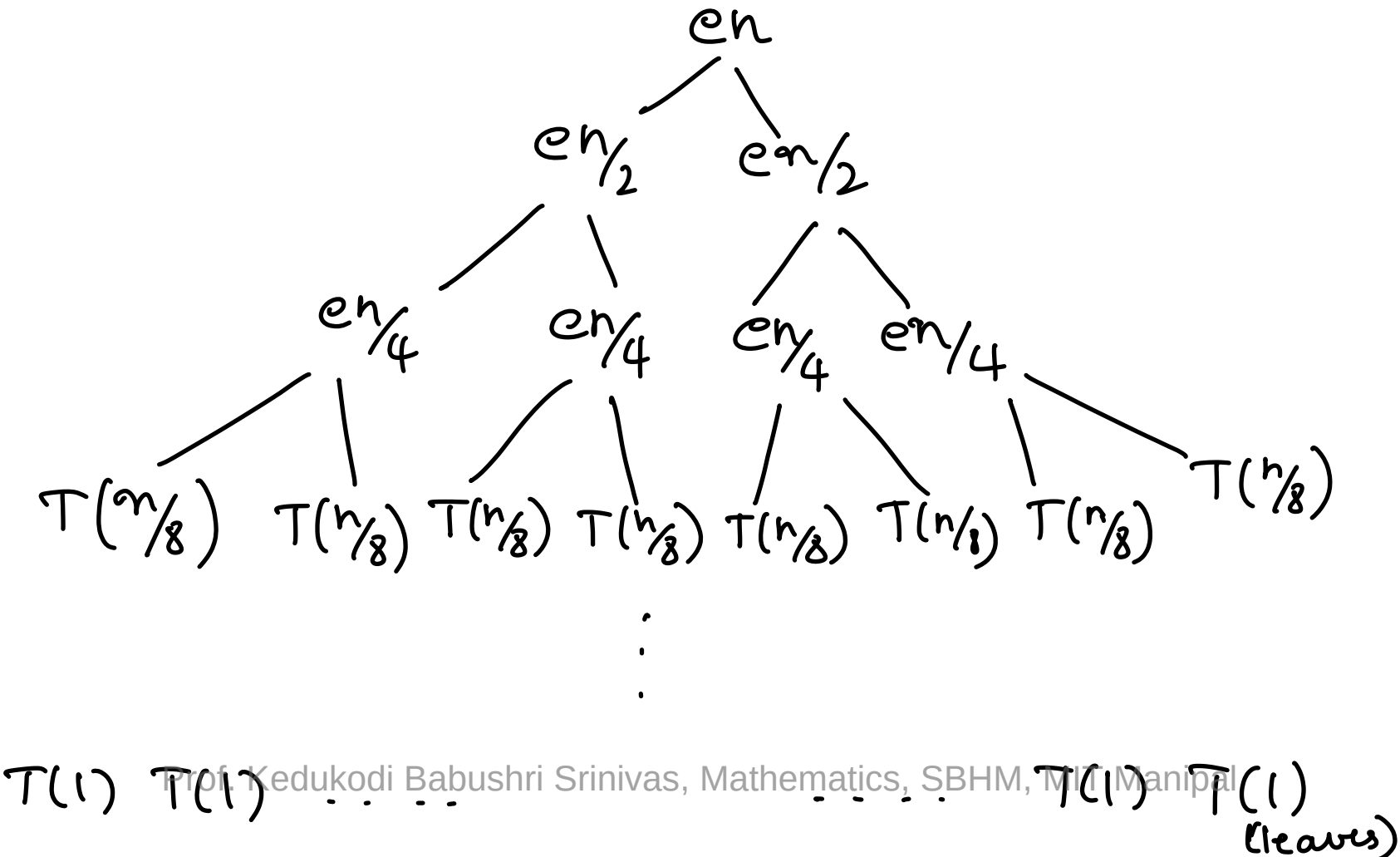
Replace n by $n/2$ in $(*)$. to get $T(n/2)$

$$T(n/2) = 2T(n/4) + cn/2$$



Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

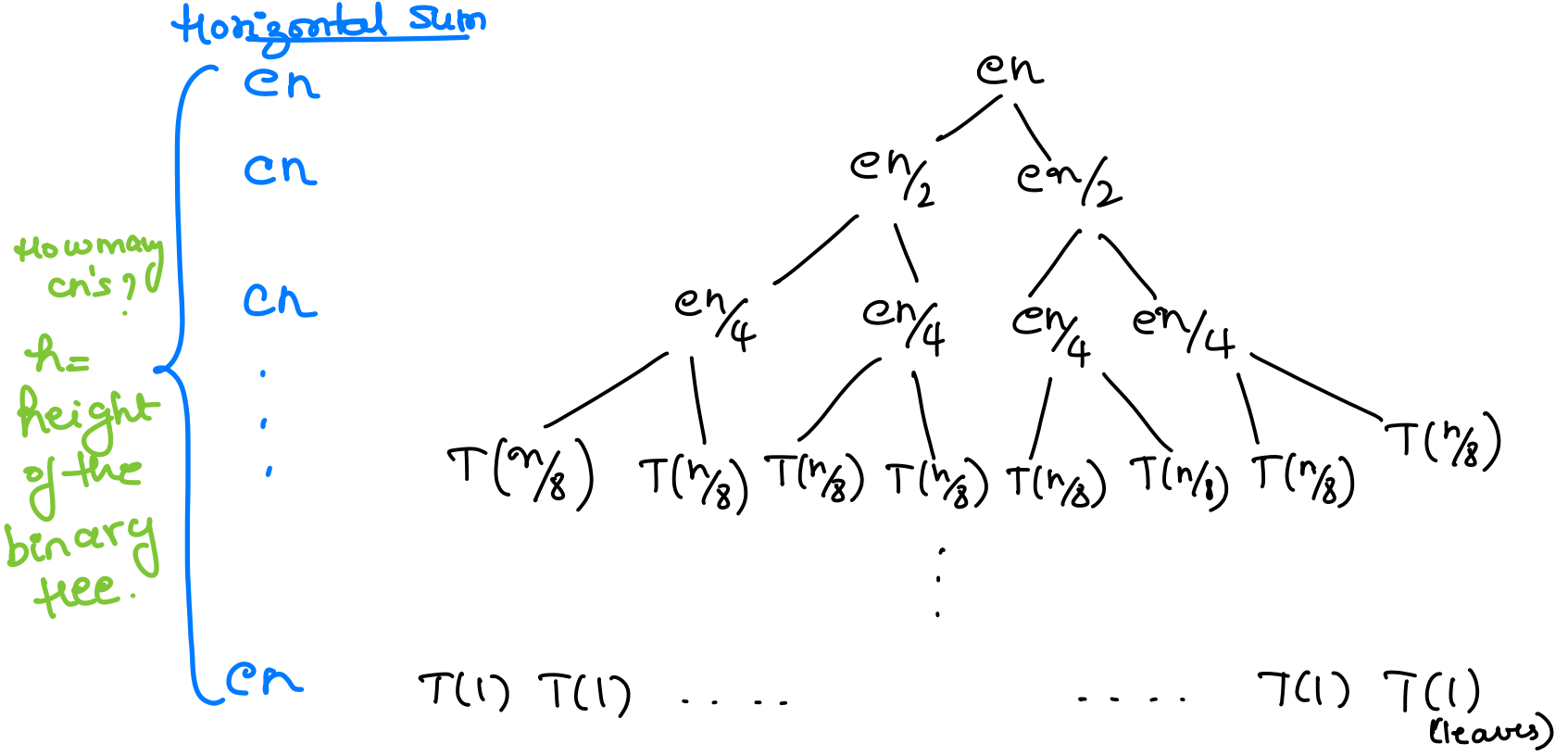

cn
cn/2
cn/2
T(n/4)
T(n/4)
T(n/4)
T(n/4)



Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

Here,

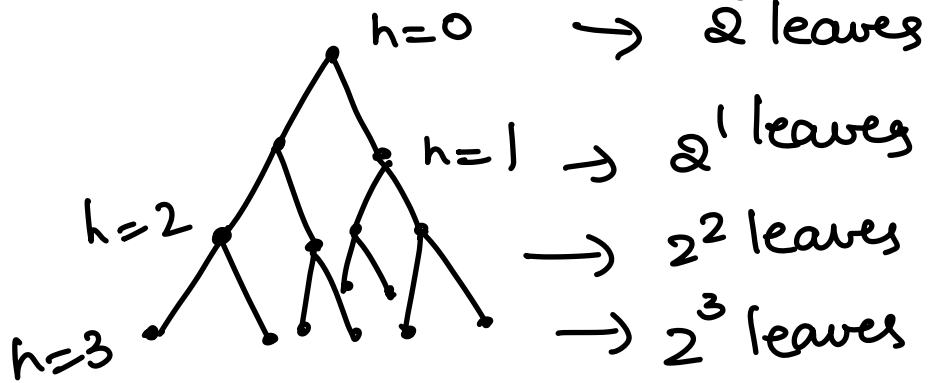
$T(n)$ = Sum of all the nodes in the tree.

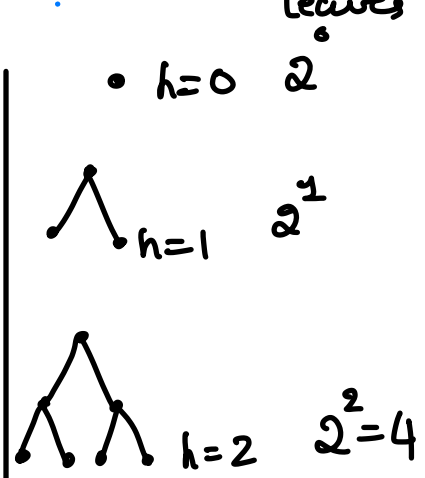

Horizontal Sum
cn
cn
cn
how many cn's ?
h = height of the binary tree.
cn
cn
cn/2
cn/2
cn/4
cn/4
cn/4
cn/4
T(n/8)
T(1)
T(1)
T(1)
T(1)
(leaves)

$\therefore T(n) = h(cn)$

Result:
$h = \lfloor \log n \rfloor$

How many leaves at each height?





Note: degree 1 vertices are called leaves.

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

The tree can have 2^h leaves at height h.

Input size $= n$

In the merge sort algorithm, every input number must be considered as sorted single numbers.

$$\therefore \quad 2^h \geq n$$

(Solving for h)

$$\log 2^h \geq \log n \qquad (\log \text{ is monotic})$$

$$h \geq \log n .$$

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal

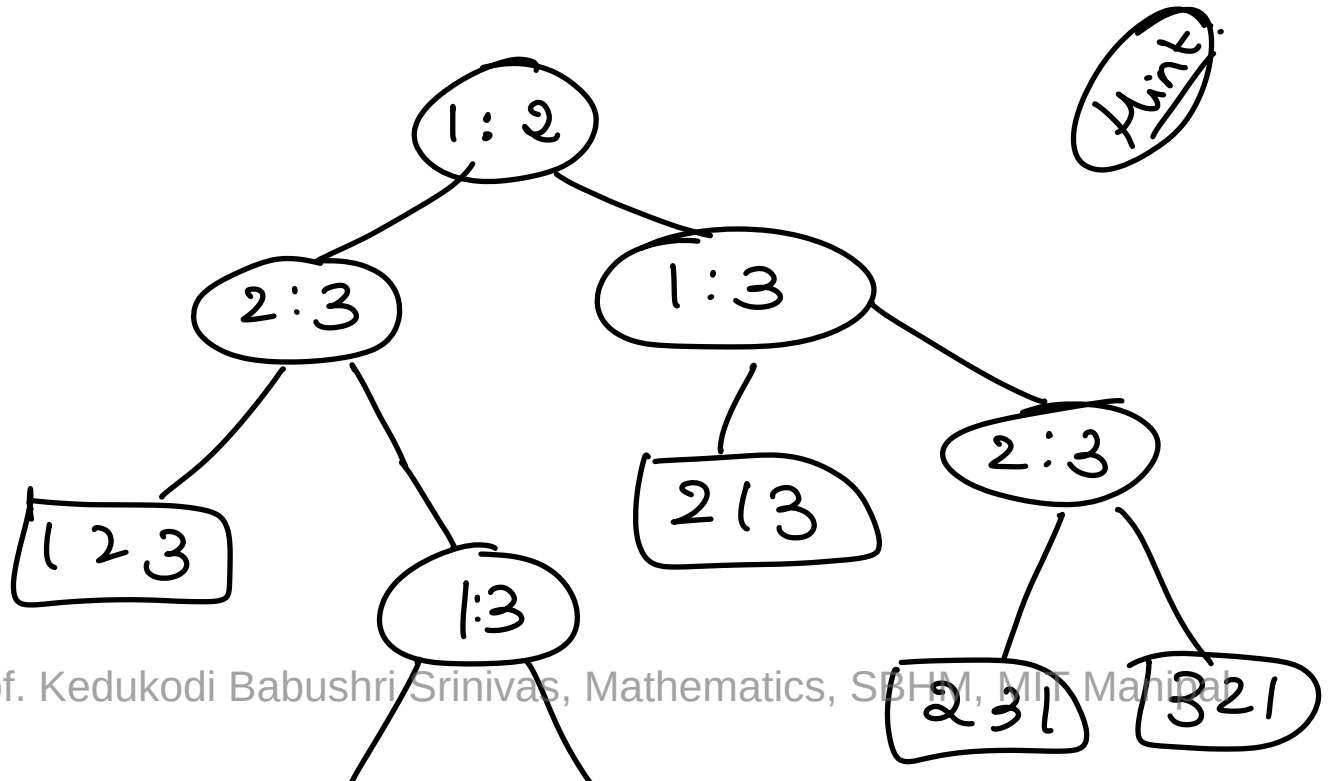
Take $h = \lceil \log n \rceil$

$\therefore T(n) = \lfloor \log n \rfloor \in \mathcal{O}(n \log n)$.

A Puzzle:

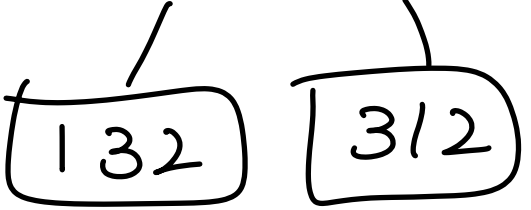
Decision - Tree Model.

Sort $\langle a_1, a_2, a_3 \rangle$



Hint:

Prof. Kedukodi Babushri Srinivas, Mathematics, SBHM, MIT Manipal



Input: 50 25 75 Output: 25 50 75